Data Scale, Tokenization and Transitive Reasoning in Two Devanagari Language Models

Karan Thayat

Language Models and Agents, Monsoon 2026 — Individual Project
Consolidated report, Phases 1–3 • 16 September 2026

This report consolidates a three-phase project that builds, pretrains and finetunes two fully independent monolingual language models: Model H for Hindi, the higher-resource language, and Model L for Nepali, the lower-resource one. Each model has its own corpus, tokenizer, vocabulary and weights, and nothing is shared between them. The two corpora supply 580,162,728 and 612,476,098 training tokens, and both models use an identical 24,562,944-parameter decoder-only transformer trained with an identical recipe, so every measured difference is attributable to the data. Held-out perplexity favours Hindi (24.91 against 36.35) while bits-per-byte favours Nepali (0.5173 against 0.4856), a disagreement caused by a 16 % difference in tokenizer fertility. After finetuning on 16,000 synthetic transitive-comparison examples, exact match rises from 0.046 to 0.9965 for Model H and from 0.0025 to 0.9985 for Model L.

1 Project at a glance

Table 1: Headline measurements for both models across the three phases. Every figure is reproduced from the JSON artifacts listed in Section 7.

	Model H — Hindi	**Model L — Nepali**
Phase 1 — corpus and tokenizer		
Documents kept	940,329	774,236
Words	392,120,217	364,030,498
Training tokens	580,162,728	612,476,098
Tokens, all splits	591,683,583	638,418,007
Manual-collection tokens	125,279,655	366,130,617
Manual share of training tokens	21.6 %	59.8 %
Tokenizer (Unigram) vocabulary	8,000	8,000
Fertility (tokens per word)	1.453	1.692
Phase 2 — pretraining		
Trainable parameters	24,562,944	24,562,944
Optimizer steps / tokens seen	12,000 / 589,824,000	12,000 / 589,824,000
Best validation loss	3.2320	3.5568
Test perplexity	24.91	36.35
Test bits-per-byte	0.5173	0.4856
Phase 3 — reasoning		
Synthetic examples (train/val/test)	16,000/2,000/2,000	16,000/2,000/2,000
Exact match, pretrained	0.0460	0.0025
Exact match, finetuned	**0.9965**	**0.9985**
Chance baseline	0.3483	0.3381

I organised the project as a controlled comparison. The intended independent variable is the resource tier of the language, and everything else is held fixed: both models are trained on Devanagari text, both tokenizers are SentencePiece Unigram with a vocabulary of 8,000 pieces, both networks use the same twelve-layer architecture and the same 24,562,944 parameters, both runs use the same optimizer settings, the same 12,000 steps, the same token budget and the same seed (20,260,812), and both are scored by the same evaluation code, and Table 1 collects the headline measurements. Section 2 covers corpus construction and tokenization, Section 3 pretraining and language-model evaluation, Section 4 reasoning finetuning and attention, and Section 5 answers the four required comparison questions.

I chose Hindi as Model H because large curated public corpora exist for it, and Nepali as Model L because comparatively little clean text is available. Sharing the Devanagari script was deliberate: it removes script and tokenizer-algorithm differences from the comparison and leaves the data as the variable of interest, but it also creates a contamination risk that Section 2.2 addresses. One complication is worth stating at the outset. Model L ended up with *more* training tokens than Model H (612,476,098 against 580,162,728), so "lower resource" here refers to availability and cleanliness rather than to raw token count.

2 Phase 1 — Data collection, cleaning and tokenizers

2.1 Corpus construction

I build both corpora with the same code, `scripts/build_corpus.py`, run twice with different configurations. Documents are ingested from the sources in Table 3, normalised by a Devanagari-aware Unicode normaliser, filtered for quality, deduplicated, and split into train, validation and test. I keep the nukta during normalisation, since stripping it merges consonants that are distinct in both languages. Filtering rejects documents on script ratio, language identification, length, symbol and digit ratios, repeated lines, repeated n-grams and mojibake. Hindi kept $940{,}329$ of $1{,}003{,}143$ documents, a keep rate of 93.7 %; Nepali kept $774{,}236$ of $879{,}912$, or 88.0 %. The rejection profile differs in an informative way. Nepali lost $31{,}028$ documents to low script ratio and $9{,}723$ to wrong-language detection, against $17{,}453$ and 758 for Hindi, and required a near-duplicate filter that removed a further $13{,}495$ documents where Hindi needed none. That asymmetry is the quality half of the resource-tier story.

	Model H — Hindi		**Model L — Nepali**	
Split	documents	tokens	documents	tokens
Train	921,582	580,162,728	743,191	612,476,098
Validation	9,325	5,789,224	15,488	13,163,074
Test	9,422	5,731,631	15,557	12,778,835
Total	940,329	591,683,583	774,236	638,418,007

Table 2: Split sizes. Splits are drawn at the document level, so no document contributes to more than one split.

Table 2 gives the split sizes. I split at the document level rather than by sentence, so no document can contribute text to more than one split, and the held-out sets are deliberately small in proportion because the training budget is what the project is short of.

Source	**Collection**	**Words**
Model H — Hindi		
`indiccorp_hi`	downloaded	317,299,220
`wikipedia_dump_hi`	manual	36,410,002
`manual_bulk_hi`	manual	17,773,162
`manual_scraped_news_hi`	manual	8,920,220
`manual_authentic_hi`	manual	7,302,413
`manual_ocr_hi`	manual	3,236,184
`manual_news_crawled_hi`	manual	1,179,016
Total		392,120,217
Model L — Nepali		
`manual_mixed_bulk_ne`	manual	188,052,248
`nepali_corpus_local`	downloaded	152,927,014
`manual_bulk_ne`	manual	23,051,236
Total		364,030,498

Rejected for	**H**	**L**
Exact duplicate	27,127	33,730
Near-duplicate	—	13,495
Low script ratio	17,453	31,028
Wrong language	758	9,723
Too short (chars)	7,697	12,114
Too short (words)	874	2,867
High symbol ratio	3,621	348
Repeated lines	1,970	151
Repeated n-gram	1,943	1,504
Long char run	881	407
High digit ratio	290	110
Mojibake	149	196
Too long	43	—
Documents seen	1,003,143	879,912
Keep rate	93.7 %	88.0 %

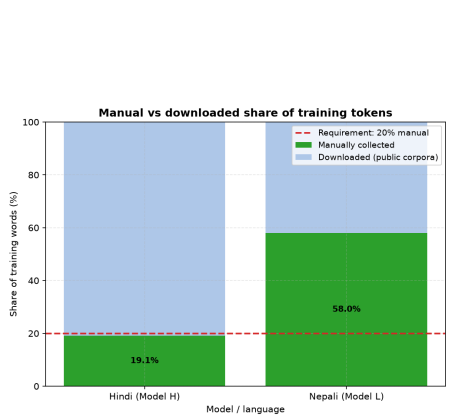
Table 3: Left: per-source word counts, and whether each source was downloaded or manually collected. Right: documents rejected by each quality filter, with the resulting keep rates.

The assignment requires at least 20 % of training tokens to come from manual collection. For Hindi I met it by running optical character recognition over eight scanned books — three Mahabharata volumes, the Rigveda, the Vishnu Puran, the Atharvaveda, a set of Upanishads and a Hindi dictionary — yielding $15{,}449$ documents and $26{,}221{,}498$ characters, together with news pages scraped and cleaned for this project and a locally processed Wikipedia dump. The outcome is $125{,}279{,}655$ manual tokens for Model H, 21.6 % of its training tokens, and $366{,}130{,}617$ for Model L, 59.8 %. I count the Wikipedia dump as manual on the course's explicit ruling that dumps qualify when the student's own extraction and cleaning pipeline processes them; after cleaning it contributes $36{,}410{,}002$ words, 49 % of the Hindi manual total. Figure 1 shows the manual share and the cleaning funnel for both languages.

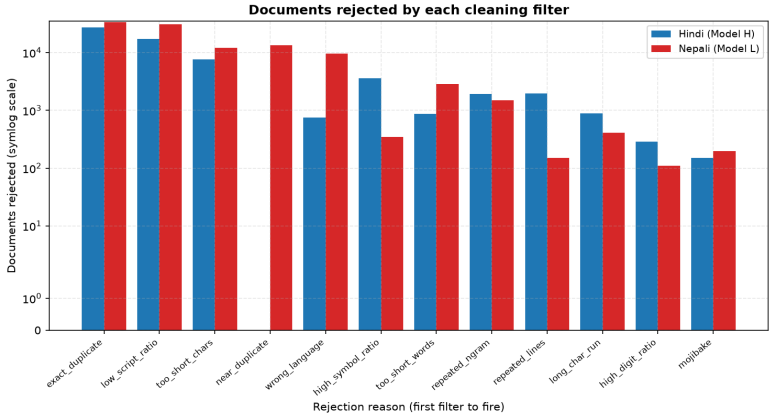
2.2 Independence of the two pipelines

Because the two languages share a script, cross-contamination would invalidate the comparison, so I check for it rather than assume it away. `scripts/validate_independence.py` reports language purity of 1.00 for both corpora, 0 documents shared between them, and 0 overlap between the train, validation and test splits within each language; it exits non-zero on any violation. The two tokenizer

vocabularies have a Jaccard similarity of 0.199, which is nonzero only because the Devanagari alphabet and the 256 byte-fallback pieces are necessarily common to both. No training document, token binary or weight is shared between the models.



(a) Manual share of collected words



(b) Documents surviving each filter

Figure 1: Phase 1 corpus construction. Nepali reaches a far higher manual share, and loses more of its input to quality filtering.

2.3 Tokenizers

Each language has its own SentencePiece Unigram tokenizer, trained only on its own training split with 8,000 pieces, `character_coverage=0.9995`, byte fallback enabled, digits split, and `normalization_rule_name=ident` because the corpus is already normalised. I chose the vocabulary size by measurement rather than assertion. At a fixed 25M-parameter budget the embedding matrix competes directly with the transformer layers, so candidate sizes from 5,000 to 10,000 were trained and scored on held-out text: moving from 5,000 to 10,000 shortens sequences by 9.9 % but costs 2.2M transformer parameters, roughly a tenth of the modeling capacity. 8,000 is the largest size at which the fertility return still justifies the embedding spend.

Vocabulary	4,000	5,000	6,000	8,000	10,000
Hindi fertility	—	1.562	1.516	**1.452**	1.411
Nepali fertility	1.918	1.827	1.763	**1.673**	1.614
Unknown rate, both	0.000	0.000	0.000	0.000	0.000

Table 4: Vocabulary sweep, measured on held-out documents. Bold marks the size I deployed for both models. Fertility falls monotonically with vocabulary size, and the unknown rate is zero everywhere because byte fallback makes any input representable.

Table 4 records the sweep, and it contains one result I decided to override. The selection rule — best held-out fertility within the permitted range — picks 8,000 for Hindi but 10,000 for Nepali. I held Nepali at 8,000 anyway. Letting the two models differ in vocabulary size would have confounded exactly the comparison the project exists to make, since a perplexity gap could then be attributed to the tokenizer rather than to the data. The cost of that decision is measurable: Nepali gives up 3.7 % on fertility, 1.673 against 1.614. I judged a controlled comparison worth more than that, but it is a choice rather than a measurement, and Section 5 depends on it.

Table 5: Tokenizer behaviour measured on 3,000 held-out documents per language. The two tokenizers are configured identically; the differences between them are produced by the data.

	H	**L**		**H**	**L**
Vocabulary size	8,000	8,000	Byte-fallback rate	0.59 %	0.99 %
Fertility (tokens/word)	1.453	1.692	Unknown rate	0.000	0.000
Characters per token	3.537	3.992	Mean piece length (ch.)	4.666	5.635
Bytes per token	8.981	10.657	Script pieces	7,230	7,710
Single-token word rate	74.9 %	59.0 %	Latin pieces	479	4
Top-1000 token coverage	73.2 %	69.8 %	Vocabulary utilisation	98.1 %	98.2 %

Two rows in Table 5 matter downstream. Fertility is 1.453 for Hindi against 1.692 for Nepali, and the single-token word rate is 74.9 % against 59.0 %. At a fixed 512-token context Model L therefore sees

fewer words in each window, and spends more of its capacity reassembling words from pieces. The unknown rate is exactly zero for both models, because byte fallback makes any input representable, so the honest measure of rare material is the byte-fallback rate: 0.59 % against 0.99 %. The Latin-piece counts, 479 against 4, show how much more code-mixed the Hindi sources are.

3 Phase 2 — Architecture, pretraining and evaluation

3.1 Model and training setup

I implemented both models from scratch in `lma/model/` as decoder-only transformers: 12 layers, 6 heads, $d_{\text{model}} = 384$, $d_{\text{ff}} = 1536$, a 512-token context, dropout 0.1, learned absolute positional embeddings and tied input and output embeddings, giving 24,562,944 trainable parameters. I wrote the multi-head attention, the positional embeddings and the causal mask myself, and `scripts/test_model.py` checks that tensor shapes are correct, that a position's logits are unchanged by any edit to later positions, and that a checkpoint resumes bit-identically. I trained with AdamW, batch 24 and gradient accumulation 4, a learning rate of 6×10^{-4} decayed on a cosine schedule to 6×10^{-5} after 500 warmup steps, weight decay 0.1, gradient clipping 1.0, mixed precision, and 12,000 steps. Each run spanned two sessions because the training platform caps a session at twelve hours; wall-clock times were 95,004 and 94,212 seconds, and the validation curves are shown in Figure 2.

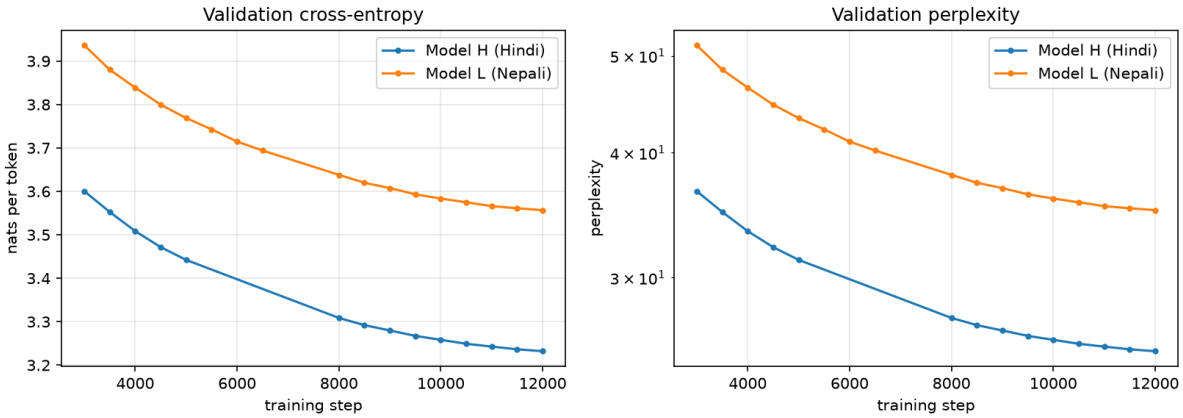


Figure 2: Validation loss during pretraining. The discontinuity in each curve is the twelve-hour session boundary, where training resumed from a checkpoint; resume was verified bit-exact.

3.2 Language-modeling results

	H	L
Cross-entropy (nats/tok)	3.2152	3.5933
Perplexity	24.91	36.35
Bits-per-byte	0.5173	0.4856
Bytes per token	8.97	10.68
Best val. loss	3.2320	3.5568

Decoding	chrF	R-L	BLEU	Rep.	Dist-2
Model H					
Greedy	13.57	10.27	1.07	0.563	0.288
$T = 0.5$	16.30	11.07	1.20	0.208	0.573
$T = 1.0$	18.71	9.82	0.76	0.002	0.910
$T = 1.5$	17.81	5.66	0.28	0.000	0.990
Model L					
Greedy	13.91	3.94	0.00	0.607	0.323
$T = 0.5$	17.25	4.58	0.00	0.174	0.693
$T = 1.0$	19.85	4.57	0.00	0.000	0.984
$T = 1.5$	19.99	2.26	0.00	0.000	0.999

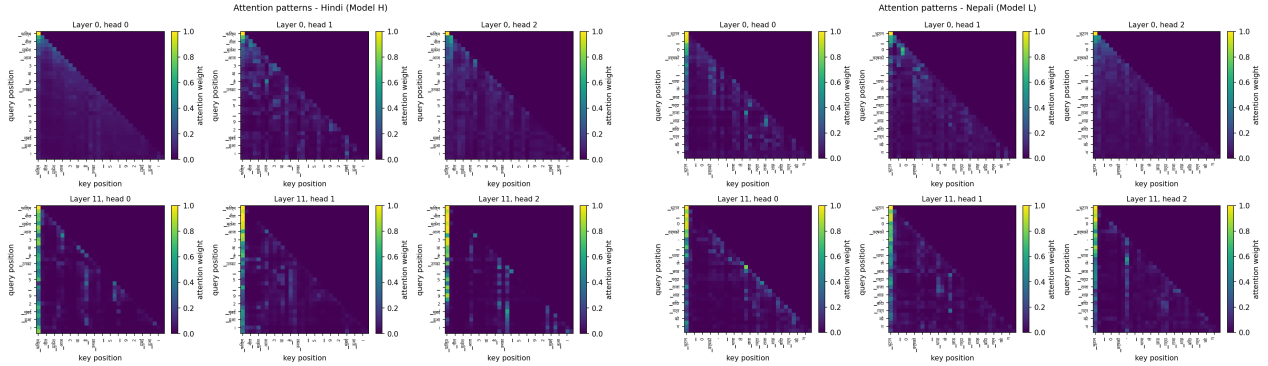
Table 6: Left: held-out language-modeling metrics on 1,638,400 test tokens per model. Right: generation quality and diversity at four decoding settings.

The two headline metrics rank the models differently, and the disagreement is the most instructive result of Phase 2. Perplexity places Model H well ahead, 24.91 against 36.35, while bits-per-byte places Model L slightly ahead, 0.5173 against 0.4856. Both are correct. Perplexity is defined per token, and a Nepali token carries more text than a Hindi one: 10.68 bytes against 8.97. Predicting one Nepali token is therefore a harder prediction covering more material, and perplexity charges Model L for that without crediting it. Bits-per-byte normalises by the underlying text rather than by the segmentation, which makes it the comparable quantity across tokenizers, and by that measure the two models are close. The generation metrics behave similarly. chrF is comparable between the models at every temperature, but ROUGE-L is not, 10.27 against 3.94 under greedy decoding, and BLEU-4 is 0.00 for

Nepali throughout. Low absolute BLEU is expected here, because these are open-ended continuations scored against a single reference; the informative quantity is the gap between the models rather than the level of either.

Greedy decoding degenerates for both models. Model H loops on उन्होंने 1990 में फिल्म 'दंगल' से बॉलीवुड में कदम रखा था। and Model L on हातखुट्टामा हात राखेर, with repetition rates of 0.563 and 0.607. At $T = 1.0$ repetition falls to 0.002 and 0.000 and both models produce fluent, correctly inflected sentences whose content is invented. At 25M parameters and 590M tokens the models have acquired the syntax and register of their corpora without acquiring reliable facts, which is also what the metric pattern in Table 6 indicates.

3.3 Attention analysis



(a) Model H **(b)** Model L

Figure 3: Pretrained attention for an early and a late layer of each model, all heads, on the same example sentence.

Figure 3 shows the heatmaps, and the per-layer statistics behind them show both models converging on the same three-stage structure. Layers 0–2 are diffuse and long-range, with Model H entropies of 2.999, 2.782 and 2.361 and mean attention distances of 14.95, 16.83 and 15.78 positions. The middle layers contract sharply: Model H falls to distances of 7.51, 7.54, 7.37 and 6.86 across layers 3–6 and reaches its lowest entropy, 1.241, at layer 7. Layers 9–11 spread out again, up to a distance of 21.27. Nepali follows the same shape with a tighter middle, reaching 4.48 at layer 5. Classifying all 72 heads gives 10 local, 48 long-range and 14 diffuse for Model H, against 15, 42 and 15 for Model L, so the two models divide labour across depth in much the same way despite never sharing data.

4 Phase 3 — Reasoning finetuning and attention

4.1 The synthetic reasoning dataset

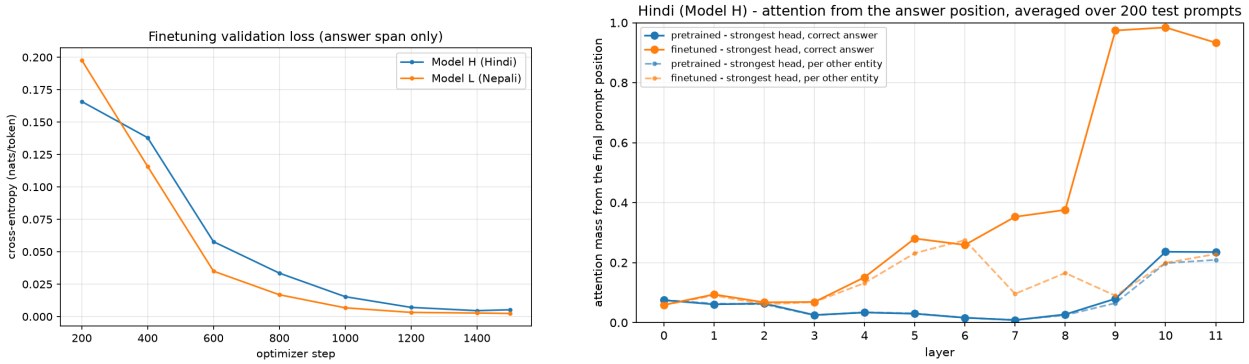
Each model is finetuned on its own synthetic dataset, which I generate programmatically with `scripts/generate_rea` so that ground truth is known by construction. Each of the 20,000 examples per language states a chain of comparisons and asks which entity lies at one end of it, for example मोहन श्याम से अमीर है। महेश मोहन से अमीर है। सबसे अमीर कौन है? उत्तर - with answer महेश. Variety comes from four comparison dimensions — age, height, speed and wealth — chain lengths of two to four entities, and two question polarities. I control leakage by reserving 20 % of entity names for evaluation only, and the checks in Table 7 report no duplicate prompts, no train–eval prompt overlap and no chain signature shared between the splits. Every answer is a single token under both tokenizers, which makes exact match an unambiguous measure.

Table 7: Composition of the two synthetic reasoning datasets, and the leakage checks run against them.

	H	L		H	L
Total examples	20,000	20,000	Unique prompts	20,000	20,000
Train / val / test	16k/2k/2k	16k/2k/2k	Duplicate prompts	0	0
Chain length 2 / 3 / 4	5,470/7,480/7,050	4,349/8,010/7,641	Train–eval prompt overlap	0	0
Dimensions	4	4	Chain-signature overlap	0	0
Shared entities	34	26	Train chain signatures	12,426	11,264
Held-out distractors	8	7	Eval chain signatures	3,337	3,139
Mean sequence tokens	22.45	24.01	Single-token answers	100 %	100 %

4.2 Finetuning

I finetune each model from its own Phase 2 checkpoint at step $12{,}000$, with its own tokenizer and vocabulary frozen and no exposure to the other language. Loss is computed on the answer span only. The settings are batch 32, a learning rate of 5×10^{-5} decayed on a cosine schedule to 5×10^{-6} after 100 warmup steps, weight decay 0.1, $\beta = (0.9, 0.95)$, gradient clipping 1.0, a maximum sequence length of 64, mixed precision, and three epochs, which is $1{,}500$ steps. The best answer-span validation loss was 0.00458 for Model H and 0.00243 for Model L, reached in 113 and 124 seconds, roughly 840 times less wall-clock compute than pretraining. Figure 4 shows both curves.



(a) Finetuning validation loss

(b) Attention on the answer entity, Model H

Figure 4: Left: answer-span validation loss during finetuning. Right: the share of a head's attention mass falling on the correct answer entity, before and after finetuning.

4.3 Reasoning results

Table 8: Pretrained against finetuned performance on the held-out synthetic test set, $2{,}000$ examples per language, with the distribution of error types.

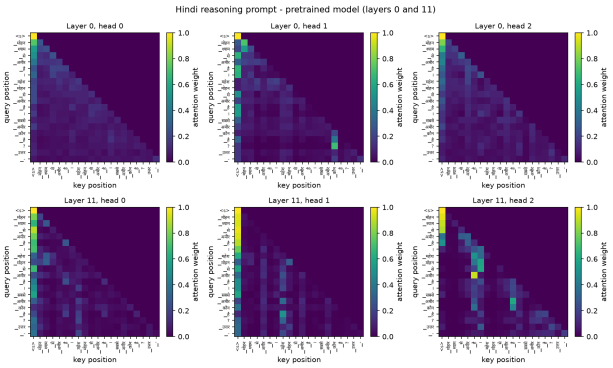
	Model H		**Model L**	
	pretrained	finetuned	pretrained	finetuned
Exact match	0.0460	**0.9965**	0.0025	**0.9985**
Constrained accuracy	0.2600	0.9965	0.2180	0.9985
Answer mean log-prob	-5.608	-0.015	-6.203	-0.009
Answer perplexity	272.57	1.0155	494.12	1.0095
Chain length 2 (1 hop)	0.0311	1.0000	0.0000	1.0000
Chain length 3 (2 hops)	0.0681	1.0000	0.0026	0.9987
Chain length 4 (3 hops)	0.0354	0.9905	0.0038	0.9975
Chance baseline (overall)	0.3483		0.3381	
Errors examined	1,908	7	1,995	3
off-chain entity	78.1 %	0 %	96.1 %	0 %
wrong entity on chain	16.6 %	100 %	3.7 %	100 %
inverted comparison	5.3 %	0 %	0.2 %	0 %

Table 8 gives the results. Both models move from near-zero to near-ceiling exact match, and both clear the per-chain-length chance baselines by margins between 0.50 and 0.75. The leakage checks and the evaluation-only entity pool rule out memorisation of test items. The error taxonomy is more revealing than the accuracy figures. Before finetuning, 78.1 % of Model H's errors and 96.1 % of Model L's name an entity that does not appear in the chain at all, which indicates that the models were not attempting the task rather than attempting and failing it; the small inverted-comparison fractions, 5.3 % against 0.2 %, suggest Model H occasionally grasped the relation and reversed its direction where Model L essentially never did. Constrained accuracy separates the two failure modes: restricting the prediction to entities named in the prompt lifts the pretrained models from 0.046 and 0.0025 to 0.260 and 0.218, so a substantial part of the pretrained failure was formatting rather than reasoning. After finetuning, every remaining error — seven for Model H and three for Model L — names an entity that is on the chain but in the wrong position, and all of them occur at chain length four.

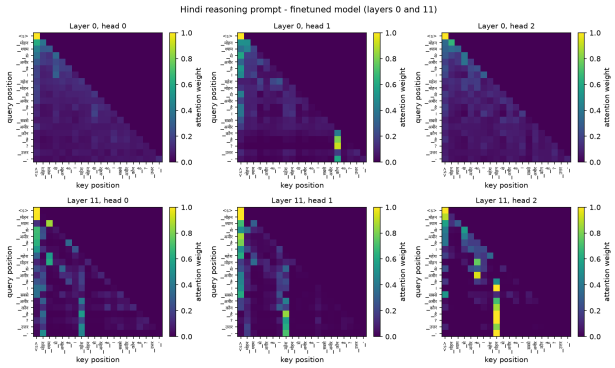
A representative Model H failure is अनिल अशोक से लंबा है। दीपक अनिल से लंबा है। अशोक सूरज से लंबा है। सबसे छोटा कौन है?, answered अशोक where the correct answer is सूरज: the model stopped one hop short of the end

of the chain. All three Model L failures involve the pair विकास and बिकास, which are spelling variants of the same name that the generator treated as distinct entities. That is a dataset artefact rather than a reasoning failure, and it accounts for the whole of Model L's residual error.

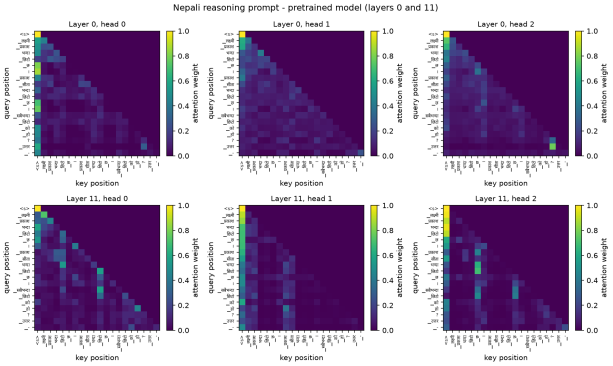
4.4 Attention before and after finetuning



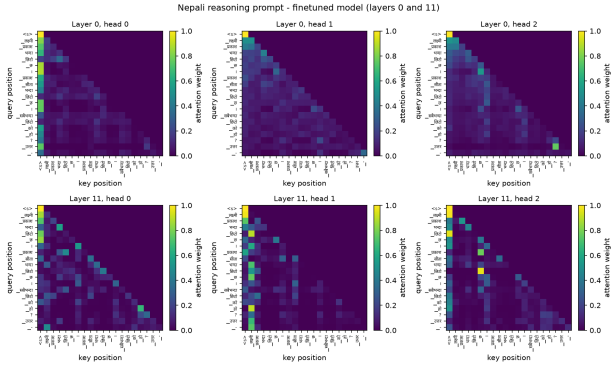
(a) Model H, pretrained



(b) Model H, finetuned



(c) Model L, pretrained



(d) Model L, finetuned

Figure 5: Attention on a comparative-reasoning prompt for an early and a late layer of each model, before and after finetuning.

Table 9: Attention statistics averaged over 200 reasoning prompts per model, before and after finetuning.

	Model H		**Model L**	
	pretrained	finetuned	pretrained	finetuned
Local heads (of 72)	34	45	29	36
Long-range heads	32	23	36	29
Diffuse heads	6	4	7	7
Layer 0 mean distance	5.35	5.04	5.59	5.56
Layer 8 mean distance	4.85	3.64	6.15	4.61
Layer 11 mean distance	5.70	5.87	6.23	6.32
Layer 0 mean entropy	1.878	1.895	1.949	1.950
Layer 11 mean entropy	1.441	1.159	1.642	1.451
Max head attention on answer, layer 9	0.080	**0.974**	0.228	**0.956**
same head, on a distractor entity	0.065	0.090	0.179	0.116
Max head attention on answer, layer 10	0.236	**0.984**	0.217	**0.980**
same head, on a distractor entity	0.198	0.199	0.179	0.424

Figure 5 and Table 9 compare attention before and after finetuning, and show three measurable changes. First, heads become more local: Model H shifts from 34 to 45 local heads and from 32 to 23 long-range ones, and Model L from 29 to 36 and from 36 to 29. Mean attention distance falls in eleven of twelve layers for Model H and ten of twelve for Model L, with the largest reductions in the middle-to-late layers (-1.22 positions at layer 8 for Model H, -1.54 for Model L). Second, late-layer attention sharpens: entropy at layer 11 falls from 1.441 to 1.159 and from 1.642 to 1.451, while layer 0 barely moves. Third, and most strikingly, individual heads in layers 9–11 become answer detectors. The most answer-focused head at layer 10 moves from 0.236 to 0.984 of its attention mass on the correct entity for Model H, and from 0.217 to 0.980 for Model L. This is selectivity rather than a general

increase in attention to entities: at layer 9, Model H's most answer-focused head moves from 0.080 on the answer and 0.065 on a distractor to 0.974 and 0.090.

5 The two resource tiers compared

5.1 Data scale and quality

Scale is nearly equal and, on raw count, favours Nepali: 612,476,098 training tokens against 580,162,728. Quality does not. Hindi retained 93.7 % of its input documents against Nepali's 88.0 %, and Table 3 shows where the difference lies: Nepali lost 9,723 documents to wrong-language detection against 758, 31,028 to low script ratio against 17,453, and a further 13,495 to near-duplicate detection that Hindi did not require. Source diversity also differs. Model H draws on seven sources including a large curated public corpus, while Model L draws on three, of which a single bulk manual source supplies 52 % of its words. The corpora differ in character inventory as well, 6,395 distinct characters against 3,093, and in Latin content, 2.09 % against 0.48 %, reflecting how much more code-mixed the Hindi web and news material is.

5.2 Language modeling against reasoning

Language modeling separates the tiers, but only on tokenizer-dependent metrics. Perplexity differs substantially, 24.91 against 36.35, and greedy ROUGE-L by more than a factor of two, 10.27 against 3.94, yet bits-per-byte and chrF are close, with bits-per-byte marginally favouring Model L. Reasoning does not separate the tiers at all once finetuned: 0.9965 against 0.9985, with Model L nominally ahead. The asymmetry follows from what each task demands. Open-ended next-token prediction over natural text rewards corpus breadth and cleanliness, which is exactly where the two corpora differ. A templated comparison task over four dimensions with single-token answers is narrow enough that 590M pretraining tokens of either quality provide a sufficient foundation, so the task saturates and its ceiling conceals any residual tier difference.

5.3 Dominant factors for the lower-resource model

Tokenizer fertility is the dominant factor. With vocabulary size, architecture, context length and training recipe all held equal, Nepali's fertility of 1.692 against Hindi's 1.453 means Model L's 512-token window covers about 14 % less text, and its single-token word rate of 59.0 % against 74.9 % means it must assemble a far larger share of words from multiple pieces. Byte fallback is secondary, 0.99 % against 0.59 %, indicating that more rare and out-of-distribution material reaches the model as raw bytes. Both are consequences of the corpus rather than of the tokenizer configuration, which was identical. Fertility is also what produces the perplexity and bits-per-byte disagreement of Section 3.2: the per-token metric measures segmentation as much as modeling ability, and once segmentation is normalised away the apparent gap largely disappears.

5.4 Why the differences arise

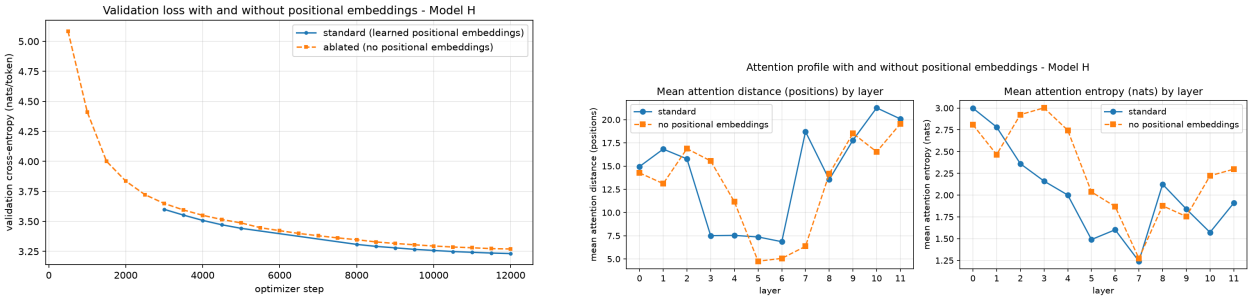
The design supports the attribution. The two models share an identical 24,562,944-parameter architecture, an identical tokenizer algorithm and vocabulary size, identical optimizer settings, step count and token budget, the same seed and the same evaluation code, and the independence checks confirm that no document, tokenizer artifact or weight is shared. Every difference reported here therefore originates in the corpora. The chain of evidence runs from the cleaning-funnel asymmetry, through the resulting fertility gap, to the perplexity difference, with bits-per-byte acting as the control that shows how much of that difference is segmentation rather than modeling. The attention statistics support the Phase 3 conclusion independently of accuracy, since answer-selective heads emerge in layers 9–11 of both models. The weaker claims are those resting on single runs: the direction of the bits-per-byte result and the small differences between the models' head classifications should be read as observations, not as established effects.

5.5 Limitations

Each configuration was trained once, so no variance estimate is available and differences of the size that run-to-run noise produces should not be read as effects. At 25M parameters and 590M tokens the absolute generation quality of both models is low, which makes BLEU-4 uninformative and limits what the qualitative samples can show. The reasoning task is templated and narrow, and its saturation means it cannot measure a resource gap. Classifying sources as manual or downloaded involves judgement. The attention results are correlational: they show where attention mass moves, not that the model's answer depends on those heads, which would require ablating them.

Table 10: Model H retrained with the positional-embedding table removed, compared against the standard model on the same data, steps, prompts and decoding settings.

	Standard	No pos.		Standard	No pos.
Trainable parameters	24,562,944	24,366,336	Val. loss @ step 3,000	3.6004	3.6493
Optimizer steps	12,000	12,000	Val. loss @ step 12,000	3.2320	3.2703
Cross-entropy	3.2152	3.2515	Greedy repetition rate	0.563	0.615
Perplexity	24.91	25.83	Greedy chrF / ROUGE-L	13.57 / 10.27	12.44 / 9.08
Bits-per-byte	0.5173	0.5231	Local / long-range heads	10 / 48	12 / 37
Layer 0 distance / entropy	14.95 / 2.999	14.27 / 2.809	Diffuse heads	14	23



(a) Validation loss

(b) Attention profile

Figure 6: Effect of removing positional embeddings from Model H. The validation-loss gap narrows as training proceeds.

6 Bonus ablation — no positional embeddings (Model H)

As an optional bonus, I retrained Model H with `use_positional_embeddings=False` and put through the full Phase 2 evaluation suite on the same data, steps, prompts, decoding settings and attention sample; Table 10 and Figure 6 report the comparison. The cost is small: perplexity rises from 24.91 to 25.83, a factor of 1.04, and bits-per-byte from 0.5173 to 0.5231. The reason the cost is not larger is that removing the table does not make the stack order-blind. A single attention layer with no positional signal does see the preceding tokens as an unordered set, but under the causal mask each position's hidden state summarises a prefix of a different length, so a second layer can recover order from prefix length; `scripts/test_reasoning.py` verifies both halves of this on small models. The narrowing loss gap is consistent with it, falling from $+0.0489$ nats at step $3{,}000$ to $+0.0383$ at step $12{,}000$. Layer 0 is the only layer with no earlier layer to reconstruct from, and it is where the loss shows: entropy falls from 2.999 to 2.809, while across all heads the diffuse count rises from 14 to 23 and the long-range count falls from 48 to 37. Behaviourally, greedy repetition rises from 0.563 to 0.615 and greedy ROUGE-L falls from 10.27 to 9.08.

7 Reproducibility and artifacts

The README gives full reproduction steps. The pipeline runs as component self-tests, corpus build, tokenizer training with the vocabulary sweep, exact token counting, the independence check, pretraining, evaluation, reasoning-data generation, finetuning and Phase 3 evaluation. Every stage is seeded with $20{,}260{,}812$, and checkpoints are resume-capable with resume verified bit-exact. `scripts/check_phase1.py`, `check_phase2.py` and `check_phase3.py` verify the deliverables of each phase and exit non-zero on a missing artifact. Table 11 lists the large artifacts held on Google Drive, all with public viewing enabled.

Table 11: Large artifacts, held on Google Drive with public viewing rather than committed to git.

Artifact	Phase	Size	Link
Hindi processed splits (H)	1	5.23 GB	`Phase1-Processed/hindi/`
Nepali processed splits (L)	1	6.61 GB	`Phase1-Processed/nepali/`
Hindi manual sources	1	1.31 GB	`Phase1-Manual/hindi/`
Nepali manual sources	1	4.36 GB	`Phase1-Manual/nepali/`
Public source corpora	1	11.56 GB	`Phase1-Sources/`
Pretrained checkpoints (H and L)	2	590 MB	`Phase2-Checkpoints/`
Finetuned checkpoints (H and L)	3	590 MB	`Phase3-Finetuned/`

Reflection

Three things in this project turned out differently from what I expected. The first was the twelve-hour session cap on the training platform, which meant neither pretraining run could finish in one sitting. What began as an inconvenience forced the checkpointing to be genuinely correct rather than approximately correct, including a test that a resumed run reproduces the original bit for bit, and that test caught problems a loss curve would have hidden.

The second was the disagreement between perplexity and bits-per-byte. I had assumed that a model trained on a cleaner and more varied corpus would win on any reasonable measure, and instead the two metrics ranked the models in opposite directions. Working out why — that perplexity is denominated in tokens, and the two tokenizers do not produce comparable tokens — changed how I read every subsequent number, and it is the single result I would keep if I could keep only one.

The third was the positional-embedding ablation, which cost far less than I expected. Reasoning about why a causally masked stack can reconstruct order from prefix length, and then confirming it on small models, taught me more about what positional embeddings actually contribute than the standard model did.

With more compute I would train each configuration several times to separate real effects from run-to-run variation, and replace the synthetic reasoning task with one that does not saturate.